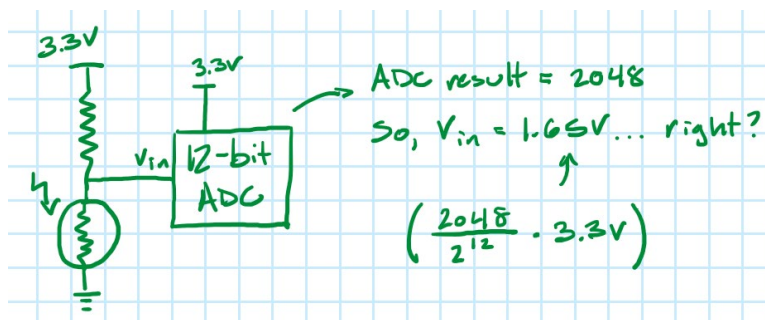


Is Your ADC a Random Number Generator?

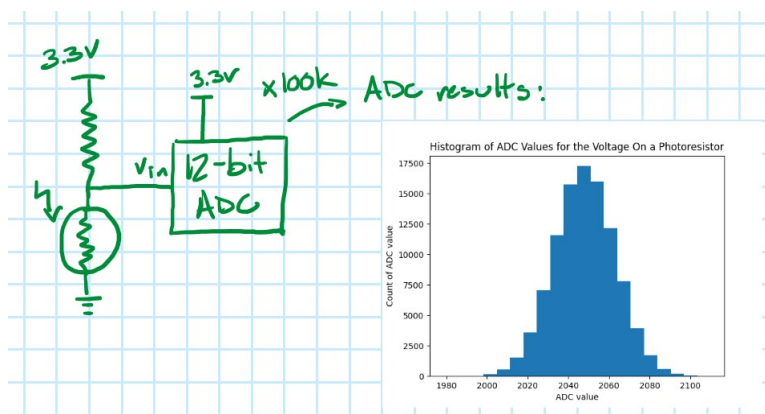
Nathan Jones

How good is your ADC? I'm not asking about how bits of resolution it has; it may come as a surprise to know that a 16-bit ADC will not always give you 16-bits of useful information! "A lot of 16-bit ADCs really operate as 7-bit ADCs and 9-bit random number generators," says Mike Stone in "Choosing an ADC"¹. In this article, I'll share with you how to determine if your ADC is generating more random numbers than useful ones, along with a few simple ways to reduce the randomness.

Let's consider a simplified setup: an ADC measuring the voltage on a photoresistor. If the 12-bit ADC below (with $V_{FS} = 3.3V$) reports a value of 2048, then does that mean that the voltage on the photoresistor is 1.65V?

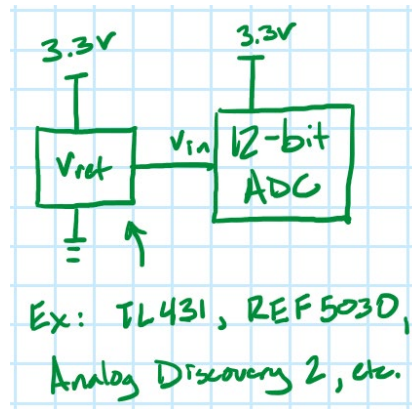


No, not really! Let me demonstrate: take a second sample. And then a third. In fact, take 100,000 samples and plot the results. What you'll see is a *distribution* of ADC values; if nothing has changed in our setup, then any *one* of these could be the true voltage on the photoresistor, with the likeliest candidate being the average value of all those samples.



Wait, what's that you say? Some of that variation could be *actual* variations in our input signal? Ah, how right you are! Let's swap out our resistive divider for a voltage reference with a known noise level and repeat the experiment.

¹ Stone, Mike. 2019. "Choosing an ADC." Adafruit Learning System. May 23, 2019. <https://learn.adafruit.com/choosing-an-adc/overview>.



Ideally the voltage reference that we use has a noise level which is less than 1/3 of the noise level we expect our ADC to have (e.g. if we expect our ADC to have ± 1 LSB of noise [which would be ± 0.8 mV using the ADC in the example above], then ideally our voltage reference has a noise level of less than 0.27 mV_{pp}); if it does, then we can say that all of our measured noise must be coming from the ADC itself, based on the fact that uncorrelated noise adds in quadrature (Equation 1).

$$Noise_{total} = \sqrt{Noise_{ADC}^2 + Noise_{Vref}^2} \quad [1]$$

(If $Noise_{Vref} = \frac{Noise_{ADC}}{3}$ then $Noise_{total} = 1.054 \cdot Noise_{ADC}$. This would mean that the voltage reference contributes just 5.4% of the total noise, a negligible amount). If it turns out that your voltage reference is noisier than 1/3 of the noise from your ADC itself, you'll just need to remember to subtract its contribution from the measured standard deviation (see Equation 2).

$$Noise_{ADC} = \sqrt{Noise_{total}^2 - Noise_{Vref}^2} \quad [2]$$

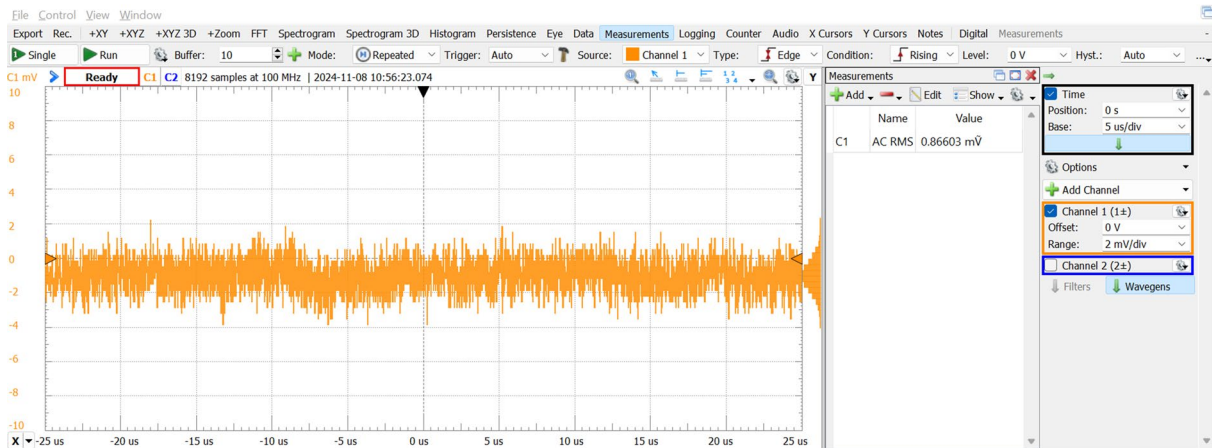
How do we *know* a voltage source's noise level?

The easiest way is to find it on the datasheet. Here's an example from the datasheet for the REF5030, showing that it has a noise level of $9 \mu\text{V}_{pp}$.

REF5030 ($V_{OUT} = 3 \text{ V}$)				
V_{OUT}	Output voltage		3	V
	Initial accuracy, standard grade		-0.1%	0.1%
	Output voltage noise	f = 0.1 Hz to 10 Hz	9	μV_{PP}

If the datasheet doesn't list a noise level (or if we just want to verify its value), we can measure it with an oscilloscope, though first we'll need to know the noise floor of our oscilloscope. We can do that by connecting one of the oscilloscope inputs to ground and measuring the RMS value of the variations in the signal you see. The figure below shows the result of doing this with my Analog Discovery 2 (AD2), demonstrating that the RMS noise is about 0.9 mV. Interesting, this value decreases (down into the μV range) if the time base is increased beyond $8 \mu\text{s}/\text{div}$, presumably because the AD2 is using oversampling and decimation to increase its effective resolution.

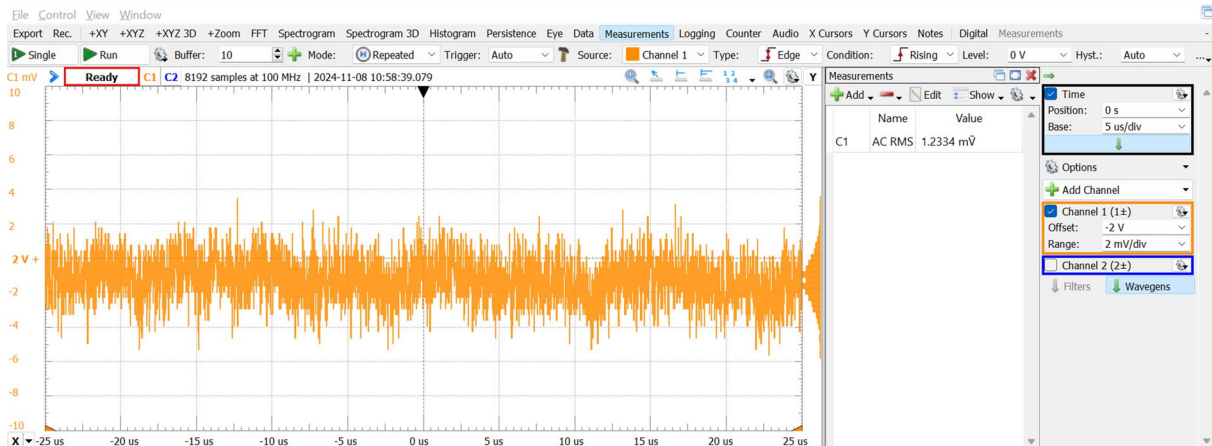
How do we *know* a voltage source's noise level?



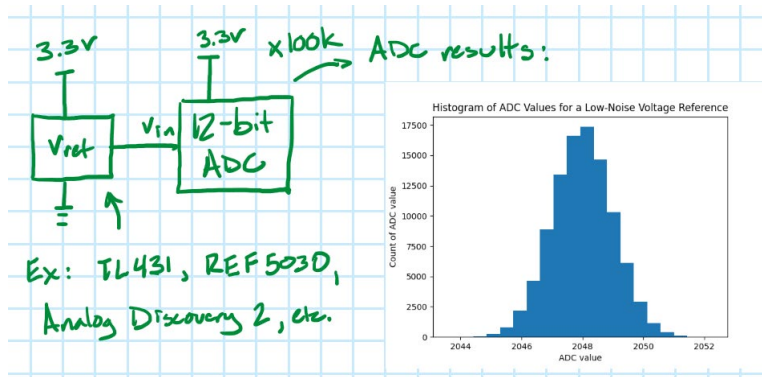
When we measure the voltage source, any variations in the oscilloscope signal are the result of the combined noise in both the voltage source and the oscilloscope itself. As long as those noise sources

are uncorrelated (they are), then $Noise_{measured} = \sqrt{Noise_{scope}^2 + Noise_{Vref}^2}$ and $Noise_{Vref} =$

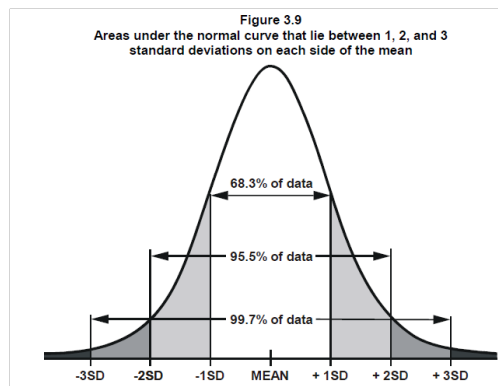
$\sqrt{Noise_{Measured}^2 - Noise_{scope}^2}$, like above. When I measured a 2V DC signal that was generated by the AD2, I determined the noise to be 1.2 mV.



This would indicate that the signal generator on the AD2 has a noise level of also about 0.9 mV. This is low enough to test a 10-bit ADC (0.9 mV is *just* less than 1/3 of an LSB for a 10-bit ADC with 3.3V full-scale reading); for any higher-resolution ADCs, we'll need to actually account for this bit of noise in our measurements.



That's much better! Notice now how the second histogram only goes out to 2052 on the x-axis (the distribution on that graph has a standard deviation of 1 LSB) while the first graph went out to 2100 (it had a standard deviation of 15 LSBs). This is called a **DC Histogram** test for an ADC and the standard deviation effectively tells us how much noise there is in our ADC: anytime we measure a value with our ADC, there's only a 68% chance that the *true* value on the analog pin is actually within ± 1 standard deviation of that value but there's a 99.7% chance that the true value is within ± 3 standard deviations of that value.



Source: <https://i.sstatic.net/jkMDV.png>

Converting this standard deviation from LSBs to volts gives us the **input-referred noise** for the ADC (Equation 3).

$$\text{Input - referred noise } [V_{RMS}] = \frac{\sigma \cdot V_{FS}}{2^N}. \quad [3]$$

(If your ADC has such little noise that all of your ADC codes *do* fall into a single bin, try lowering the ADC reference voltage. The noise level will stay the same but will take up more ADC codes than before, allowing you to find a proper value for the input-referred noise.)

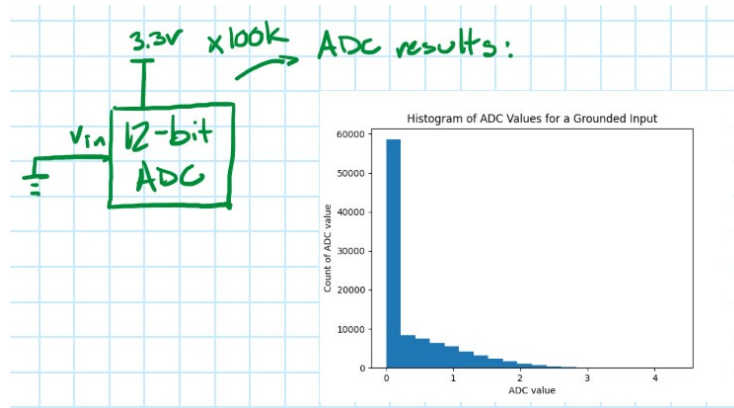
The **effective resolution** and the **noise-free code resolution (NFCR)** of an ADC represent the extent to which this noise limits our ADC resolution, which are calculated by dividing the full range of ADC codes into bins that are only as wide as either one or 6.6 standard deviations.

$$\text{Effective resolution} = \log_2 \frac{2^N}{\sigma} \quad [4]$$

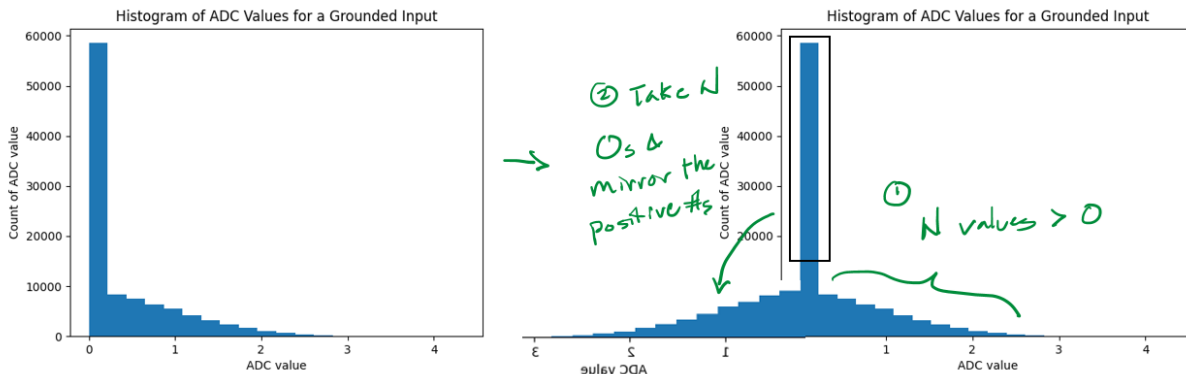
$$\text{Noise-free code resolution (NFCR)} = \log_2 \frac{2^N}{6.6\sigma} \quad [5]$$

Essentially, we have fewer actual bits in our result because there's an increasing chance that the last few bits in our ADC result are wrong or noisy.

A slightly more straightforward version of the DC Histogram test involves simply grounding the input of the ADC and then creating a histogram of the ADC results, like we did for our oscilloscope.



The standard deviation of this histogram is *also* a measure of the ADC noise (minus any contributions from noise in the ADC reference voltage, which won't affect readings of 0V), but it's optimistically low. The ADC probably reports values less than 0V as just 0, so we are only seeing *half* of the true noise distribution. I think, in this case, you could *approximate* the ADC noise if you assumed that the distribution was symmetric about its average value, changing enough of the median values to negative numbers so that the distribution became symmetric and then calculating the standard deviation of *that*.



Okay, so what do we do about all this noise? If the resolution of your ADC is still within your project requirements (lucky you!), then just keep in mind that every ADC measurement comes with some uncertainty.

- When comparing two ADC values, treat them as being equivalent only if they're within 6.6σ of each other (and you want to be 99.7% confident in your comparison) or if they're within σ of each other (and you can allow yourself to be only 68% confident).

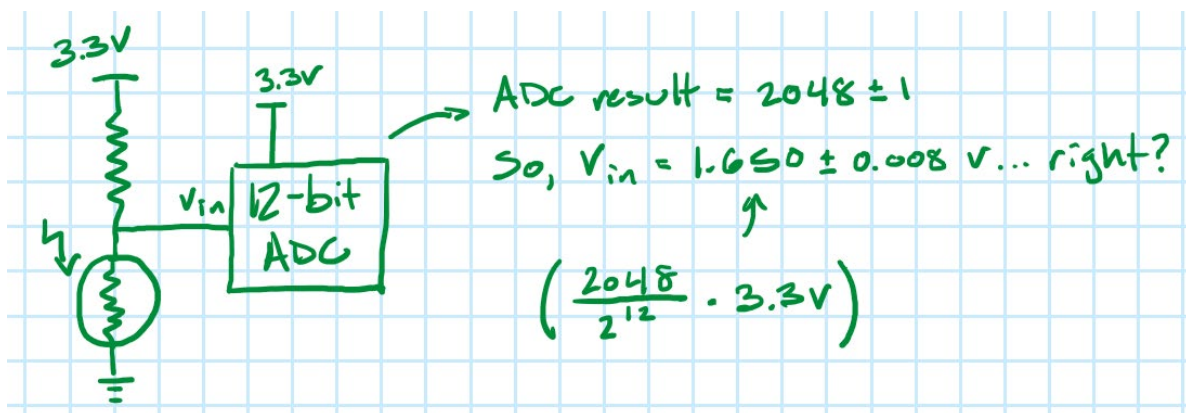
- When comparing an ADC value to a threshold, add some hysteresis to the threshold comparison (like a thermostat does) to avoid the embedded system from re-triggering above and below that threshold just based on noise.

If it turns out that your ADC *doesn't* meet your project requirements anymore, there are a few simple things to try to improve its performance. On the hardware side, these include:

- Using a lower-noise reference (such as the REF5030 [Digi-Key part number [296-27911-1-ND](#)] or TL431 [Digi-Key part number [5272-TL431-ND](#)]) for AVCC and AREF
- Ensuring minimal crosstalk between analog and digital signals on your PCB
- Turning off the CPU during ADC conversions (if your MCU supports it), to reduce digital noise

On the software side, you can oversample your input signal (i.e. take the average of N samples) to reduce noise, though this will clearly reduce your maximum sampling rate. (In fact, you could even use a technique called “oversampling and decimation”² to *increase* the resolution of your ADC by an amount related to the number of times you oversample your input!)

Okay, so let's assume the standard deviation of a DC Histogram test for our initial setup is 1 LSB. If the ADC reads 2048, *now* does that mean (there's a 68% chance) the true value is 1.650 ± 0.008 V?

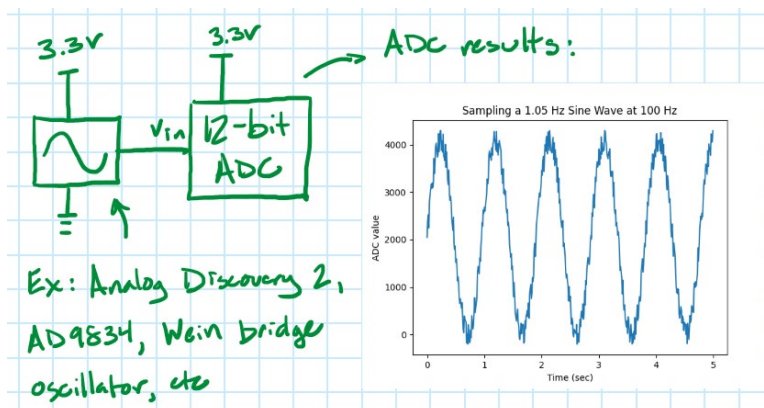


Closer, but still no! That's because all ADCs will struggle with measuring a rapidly changing input signal at some frequency and this results in imprecise measurements (i.e. noise). Surprisingly, the frequency at which that occurs may be well below the maximum sampling rate for your ADC! This means that a 12-bit ADC with a sampling rate of 1 MSPS (mega-samples per second) *may not actually have* 12-bits of resolution for input signals near 500 kHz; the resolution of that ADC may fall to 6-bits or lower when the input signal frequency is that high. If we're trying to measure a signal on the photoresistor that's above 10 Hz, our ADC may not actually have 12-bits of resolution.

To determine how well the resolution of our ADC holds up with higher input frequencies, we'll do something very similar to what we did in the DC Histogram test: we'll send in a known-good signal (this time, a sine wave) and measure how closely the ADC values match that known-good signal. (How do we

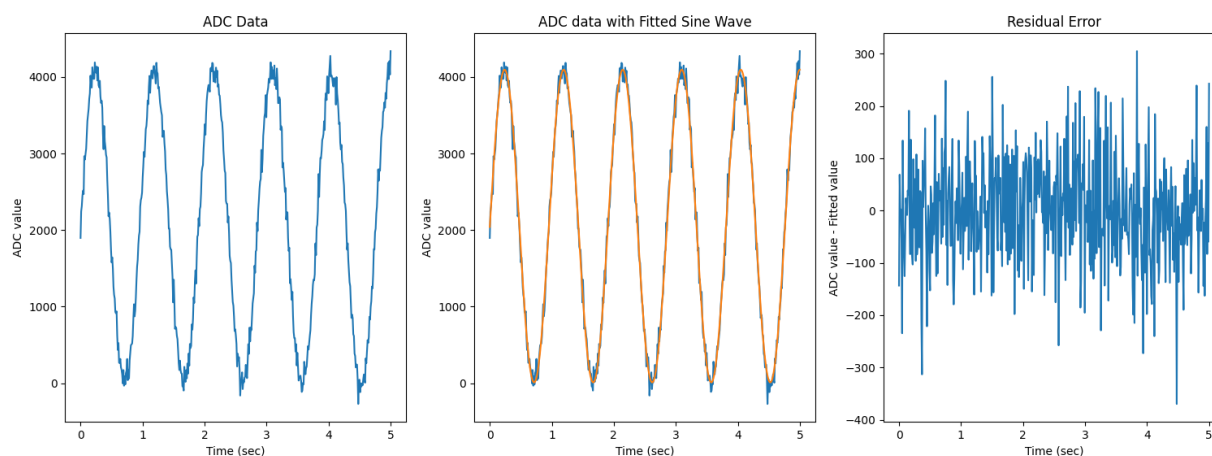
² "AVR121: Enhancing ADC Resolution by Oversampling Features." n.d. Accessed November 14, 2024. <https://ww1.microchip.com/downloads/en/appnotes/doc8003.pdf>.

know our sine wave is “good enough”? We’ll discuss that after we learn how to perform our test; it will make more sense that way.)



For the sine wave, we can use a function generator (like the ones found on many USB oscilloscopes), a DDS chip (like the AD9834 [Digi-Key part number [AD9834BRUZ-REEL7CT-ND](#)]), or we can make our own sine wave generator (based on a Wien bridge oscillator or by using a MAX7400 from Analog Devices [Digi-Key part number [MAX7400CSA+-ND](#)] to lowpass-filter a square wave³, for example). An important note is that you should only ever use input frequencies for this test that are *not* integer sub-multiples of your sampling frequency (e.g. don’t use an input frequency of 1 kHz if your sample rate is 100 kHz). This is to avoid erroneously sampling the sine wave in the same locations on every cycle. Additionally, to maximally stress your ADC, you should set your input sine wave’s amplitude to the largest value that doesn’t “clip” the output (e.g. something slightly less than 1.65V for the ADC in the examples above).

After collecting the ADC results, we’ll fit the data to a sine wave and calculate the residual error. You can perform a sine wave fit with almost any amount of data, but you’ll get the best results if you have at least five full periods of the input sine wave. You’ll likely need to help out your sine-fitting algorithm by giving it initial guesses for the amplitude, frequency, phase shift, and offset.



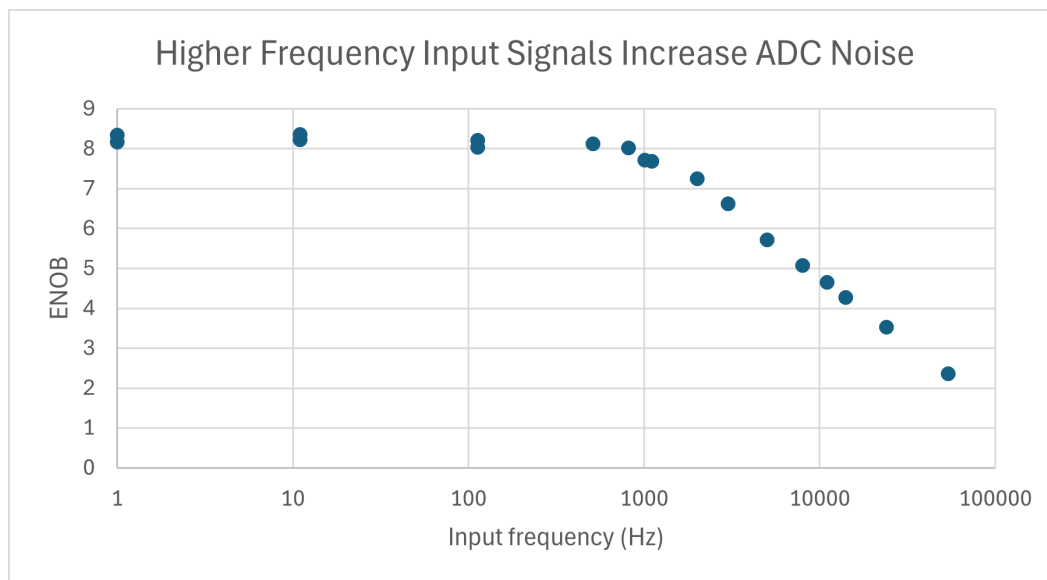
³ “Digitally Controlled Sine-Wave Generator.” 2023. Analog.com. Analog Devices, Inc. February 20, 2023. <https://www.analog.com/en/resources/technical-articles/digitally-controlled-sine-wave-generator.html>.

This test is called the **sine wave fit** test for an ADC and the RMS value of the residual error is a measure of how noisy your ADC is at that specific input frequency. At lower frequencies, the RMS noise should match the input-referred noise from your DC histogram test, but at higher frequencies it will almost certainly degrade as a result of those frequency-dependent noise sources I mentioned above.

We can use the RMS noise value from the sine wave fit test to calculate the **effective number of bits (ENOB)** of our ADC (Equation 6), which is the number of bits of an ideal ADC whose only noise source, quantization noise, has the same RMS value as the noise in our ADC.

$$\text{Effective number of bits (ENOB)} = \log_2 \frac{2^N}{\sqrt{12} \cdot \text{Error}_{RMS}} \quad [6]$$

This value is specific to the frequency of the input signal that was used. To get a more complete picture of your ADC, you should repeat this test at frequencies all the way up to your highest input signal of interest, or up to half of your maximum sampling rate. You may find that your noise goes way up (and your ENOB way down!) at frequencies well below half of your maximum sampling rate! For example, I did that exact series of tests on the ADC in an STM32F042K6 and plotted the results (below).



Even though the ADC on the STM32F042K6 microcontroller can sample at 1 MHz, the resolution falls off above 1 kHz; if you were intending to measure a 10 kHz signal with this ADC, you'd only have about 5 bits of resolution at that point! (Notice, also, that the maximum ENOB is around 8.3; this is due to ~10 LSBs of noise even at DC, which reduce the effective resolution of the ADC from 12 bits to around 8.7 bits right off the bat. These tests were conducted on an unmodified Nucleo-F042 development board; results would be much better if I'd used any of the techniques mentioned above ["Okay, so what do we do about all this noise?"].)

How do we *know* a sine wave's noise level?

As with the voltage reference, if we don't know the noise level of the sine wave *a priori* (i.e. if it's not available on a datasheet) then we'll need to measure it with an oscilloscope. We'll use the technique above, sine wave fitting, to characterize the noise from our sine wave generator.

1. First, determine the noise floor of your oscilloscope by connecting its input to ground and calculating the RMS value of the output (as described above in "How do we *know* a voltage source's noise level?").
2. Then, measure the output from your sine wave generator, fit the results to a perfect sine wave, and calculate the RMS value of the residuals. You can then use Equation 2 to determine the noise of your sine wave generator (using the oscilloscope's noise floor as an approximation of its AC noise).

Ideally the noise level of the sine wave is less than 1/3 the noise level that you expect from your ADC (in other words, the sine wave is "spectrally pure"), which would make its contribution to the measured noise negligible. If the noise level of the sine wave is higher than that, though, you'll just need to remember to subtract it from the measured noise that we get from calculating the RMS values of the residual error.

This test should be repeated for every input frequency you intend to use to test your ADC (even your sine wave generator may have frequency-dependent noise sources!).

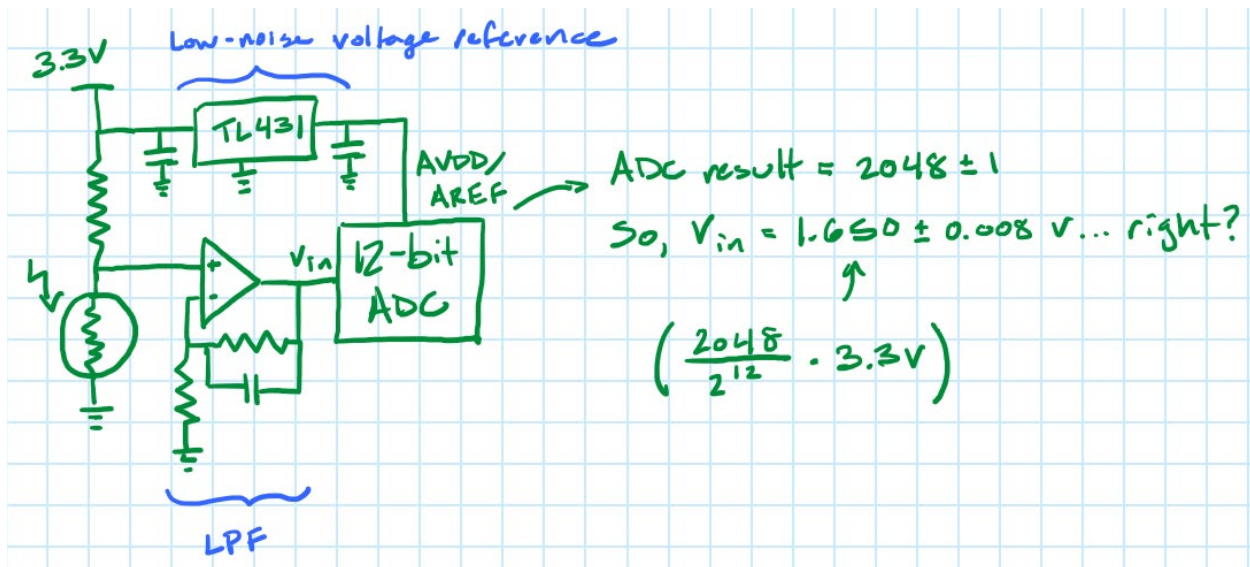
Okay, so what do we do about all this noise? First, I would recommend putting a low-pass filter (LPF) in between your signal source and your ADC, with a cutoff frequency just above your highest signal of interest, to help limit high frequency noise (this is a "good thing to do", in general, for any data acquisition system). If that highest frequency signal of interest is below the "knee" in the graph of ENOB vs Input frequency, then you haven't really lost any resolution. If you're trying to measure signals above that "knee", there are a few things you can try to improve that resolution at higher frequencies. These include:

- Ensuring that the output impedance of the device that's connected to the analog pin is very low, possibly inserting an op-amp for buffering
- Adjusting the ADC clock or its sample/hold time to be as fast as possible, while still giving the internal capacitor enough time to charge
- Using a low-jitter ADC clock source and configuring the system to take ADC samples at regular intervals

Unless you're looking for a signal in a very narrow band of frequencies, the most straightforward thing to do next is to assume that your actual input signals have equal frequency content from DC up to the cutoff frequency of your LPF and therefore that the RMS noise in every ADC measurement is the average RMS noise you measured using the sine wave fit test from DC up to your cutoff frequency. If we wanted to measure broadband signals up to 10 kHz using the STM32F042K6 from above, then we would be best to operate as though our ADC only had an ENOB of around 6 bits for each measurement. This value may very well be lower than the one you determined using the DC Histogram test, indicating that you have

less certainty in each of your ADC measurements if there's a possibility that they have higher frequency content.

Okay, so let's assume we've added a LPF in between our sensor and our ADC such that the RMS noise from DC up to the cutoff frequency is only ± 1 LSB. If the ADC reads 2048, *now* does that mean (there's a 68% chance) the true value is 1.650 ± 0.008 V?



As crazy as it sounds, *still not yet!* Although we've effectively characterized the *noise* in our ADC, we haven't yet characterized its *error*. After our testing, we can only be confident that two ADC values are the same or different, but we don't know whether an ADC code of 2048 corresponds to *exactly* 1.65 V or to something else. Gain, offset, and differential non-linearity errors (among others) could mean that converting from an ADC code to an actual voltage is more complicated than using $V_{in} = \text{ADC result} \cdot \frac{V_{FS}}{2^N}$. But that's a topic for another article!

If you're made it this far, I sincerely appreciate your time and attention! Happy hacking!

References

- [Characterizing the Raspberry Pi Pico ADC](#)
- [Understanding Data Converters \(Texas Instruments\)](#)
- [Understand SINAD, ENOB, SNR, THD, THD + N, and SFDR so You Don't Get Lost in the Noise Floor \(Analog Devices\)](#)
- [The Good, the Bad, and the Ugly Aspects of ADC Input Noise--Is No Noise Good Noise? \(Analog Devices\)](#)
- [Getting the most out of the SAM D21's ADC \(Thea Codes\)](#)
- ["Analog-to-Digital Converter Testing" \(Kent Lundberg\)](#)
- [Dynamic Tests For A/D Converter Performance \(Texas Instruments\)](#)
- [How to optimize the ADC accuracy in the STM32 MCUs \(STM\)](#)
- [Enhancing ADC resolution by oversampling \(Atmel\)](#)
- [Fundamentals of Precision ADC Noise Analysis \(Texas Instruments\)](#)
- [ADC Gain and Offset Error Calibration on ARM® Cortex®-M0+ Based MCUs \(Microchip\)](#)

- [Sine wave generation via low-pass filtering a square wave](#)
- [Understanding the impact of digitizer noise on oscilloscope measurements \(EE Times\)](#)
- [Understanding Effective Number of Bits \(Robust Circuit Design\)](#)